Orange Research

Shape the future and value innovation

Challenge EURO/ROADEF 2026

Keep the flow!

EURO THE ASSOCIATION OF EUROPEAN OPERATIONAL RESEARCH SOCIETIES

ROADEF SOCIÉTÉ FRANÇAISE DE RECHERCHE OPÉRATIONNELLE ET D'AIDE À LA DÉCISION

Problem Description

Challenge Organizers

Amal Benhamiche, Yannick Carlinet, Morgan Chopin, Eric Gourdin, Nancy Perrot

Contents

1 Introduction **1**

2 Preliminaries **1**

3 Problem description **3**
- 3.1 Decisions . . . 3
- 3.2 Constraints . . . 4
- 3.3 Objective . . . 5
- 3.4 Problem definition . . . 5

4 Example of instance **5**
- 4.1 Network . . . 5
- 4.2 Demands . . . 6
- 4.3 Intervention Scenario . . . 6
- 4.4 Solutions . . . 7

5 Input/Output File Formats **7**
- 5.1 Input files . . . 7
  - 5.1.1 Network Topology: `toy-net.json` . . . 8
  - 5.1.2 Traffic Matrix: `toy-tm.json` . . . 9
  - 5.1.3 Interventions Scenario: `toy-scenario.json` . . . 9
- 5.2 Output . . . 10
  - 5.2.1 Segment Routing Paths (SR-paths): `toy-srpaths.json` . . . 10

Bibliography **11**

A Table of notations **11**

1 Introduction

Orange is managing many networks, including large scale national and international IP/MPLS (Internet Protocol/Multiprotocol Label Switching) backbones with thousands of routers and links and about tens of thousands of traffic demands per year. With the arrival of next-generation networks enabled by disruptive technologies such as Software-Defined Networking (SDN) and Network Virtualization, network managers have to reconsider classic *Traffic Engineering* (TE) challenges and appropriate solutions through a different lens. These challenges include the adaptation of networks to accommodate the ever-growing volume of traffic and the multiplicity of user services and contents, all while maintaining high quality of service (QoS). Like for other telecommunications operators, TE is therefore a critical field at Orange as it allows to better tune network parameters so as to make a more efficient use of the existing infrastructure and resources. The purpose is mainly to avoid congestion and hence to improve the QoS experienced by users of the multiple services supported by the network. A standard design rule requires the network to remain 100% functional whatever single link or node failure might occur. The term failure should be understood broadly, covering cases where a link becomes unavailable either due to unforeseen events or deliberate interventions by a network operator. Moreover, today's geopolitical tensions and accelerating climate change increase the need for networks that are not only secure but also resilient to multiple router or link failures caused by natural disasters or acts of sabotage.

In this context, the goal of this challenge is to design an optimization algorithm that, given an IP/MPLS network and a traffic matrix (that is, a set of projected values based on current measured volumes of traffic between given origin/destination pairs), computes a routing scheme that maintains acceptable network load, even in scenarios of network equipment unavailability.

The proposed routing scheme relies on the so-called Segment Routing (SR) protocol, a network technology proposed by IETF [1] which recently attracted much attention in both telecommunication networking and Operations Research communities. Indeed, the packets in a IP/MPLS network are traditionally routed along the shortest paths from the origin to the destination according to a set of arc weights set by the network administrators. While this approach is easy to implement in practice, it comes with several limitations such as the complexity of finding set of link weights that induce shortest-paths with the least possible network congestion while remaining robust against scenarios where multiple network components become unavailable.

Segment Routing was designed to palliate this issue by enabling the possibility to route packets over non-shortest paths without extensive modifications to the network. More precisely, each packet entering the network is assigned to a so-called *segment path*, which is a sequence of routers referred to as *node segments* or *waypoints* that the packet must visit, one after the other in the network before reaching its final destination. Between two waypoints, traditional shortest path-based routing is used. By encoding routing instructions directly into the packet header, SR allows packets to deviate from the single shortest paths from origin to destination, and hence allowing greater flexibility with minimal additional implementation costs for network administrators.

The design of sets of segment paths that account for multiple routers or links unavailability is a very challenging yet crucial optimization problem. Addressing this is critical to ensure *stability* and enable deployment in modern, large-scale IP/MPLS networks.

2 Preliminaries

In simple terms, the problem consists of determining the segment path for each demand in order to balance the network load. Prior to providing the formal definition of the optimization problem, we introduce the necessary preliminary notations and definitions.

Network A *network* is a tuple $(G = (V, A), \omega, c)$ where G is a directed graph of n vertices with vertex set $V = \{v_0, v_1, \ldots, v_{n-1}\}$ and arc set $A \subseteq \{a_{i,j} : (v_i, v_j) \in V \times V\}$, $\omega : A \to \mathbb{Q}$ is a weight function used to compute the shortest paths in G, and $c\colon A \to \mathbb{Q}$ is a capacity function that represents, for each arc $a \in A$, its bandwidth $c(a)$ (given in Mbits or Gbits in practice), that is, the maximum amount of traffic throughput that it can accommodate. The terms "arc" and "link" are used interchangeably throughout this document.

Forwarding graph & ECMP Let $(G = (V, A), \omega, c)$ be a network. The *forwarding graph* from a *source* $u \in V$ to a *target* $v \in V$ is the subgraph of G containing all arcs that belong to any shortest path (according to weights ω) from u to v. It is denoted $FG(u, v)$. When $FG(u, v)$ is not a simple path, the *Equal-Cost Multi-Path* (ECMP) mechanism is activated: the incoming flow in a vertex of $FG(u, v)$ is evenly divided between the outgoing arcs of this vertex in $FG(u, v)$ (see Figure 1).

Segment routing path A *segment routing path*, or *segment path*, in a network $(G = (V, E), \omega, c)$ is a succession of forwarding graphs such that the target of the previous forwarding graph coincides with the source of the next one. A segment path is denoted by $\langle s, w_1, \ldots, w_\ell, t\rangle$ where s is the source of the segment path, t the target, and $w_1, \ldots, w_\ell \in V$ are the *waypoints* in that order. The source and target do not count as waypoints. The case where there is no waypoint (i.e. $l = 0$) is possible; it means that the demand is simply routed on the shortest paths from s to t.

Segment In this context, we define a *segment* as a pair of successive nodes in a segment path. These nodes are either the source, target or waypoints included in a segment path. For instance, $(s,\ w_1)$, $(w_1,\ w_2)$, $\ldots$, $(w_\ell,\ t)$ are the segments composing the segment path $\langle s, w_1, \ldots, w_\ell, t\rangle$. Each segment (u, v) of a segment path is thus associated with the forwarding graph $FG(u, v)$. A typical maximum number of segments in a segment path is between 4 and 10, depending on the underlying protocol and the router technology.

Demand A *demand* on a network $(G = (V, E), \omega, c)$ is a couple (s, t) where the *terminals* $s, t \in V$ are respectively the *source* and *target* of the demand. Each demand is associated with a traffic volume, i.e. the size of the flow that needs to be routed from the source to the target. If the traffic volume is 1, it is referred to as a unit demand. The volume can vary in time, therefore the traffic volume is a function denoted $\nu : D \times T \to \mathbb{Q}$ with T the discrete set of time periods and D the set of demands, referred to as the traffic matrix.

Routing scheme Given a network $(G = (V, E), \omega, c)$ and a set of k demands $D = \{(s_i, t_i) : i = 1, \ldots, k\}$ on G, a *routing scheme* for D is a set P of k segment paths $\{p_d : d \in D\}$ such that p_d is a segment path from s to t associated to the demand $d = (s, t)$.

Time horizon The routing scheme has to be decided in advance and over a given time period (from a few days to a couple of weeks). We assume that the traffic demand values can be estimated through forecasting for each time period, whereas the network state (resources status, utilization and availability) depends on the scheduled maintenance interventions. We denote by $T = \{0, 1, \ldots, h-1\}$ the set of time periods, and $h \geq 1$ the routing planning horizon. For example, $T = \{0, 1, \ldots, 6\}$ for a daily planning over a week and $T = \{0, 1, 2, 3\}$ for a weekly planning over a month. The time step 0 is a special case because it represents the nominal situation, when the network runs without any failure. Therefore, we also denote $T^* = T \setminus \{0\}$.

Interventions An intervention in the network (for maintenance, or other reason) causes a link or a node to be turned down for a certain time. Functionally, it is the same as a failure, except it can be planned in advance. In the following, we consider only links turned down, because if a node is down, it is functionally the same as if all connected links to this node are down.

Intervention scenario An *intervention scenario* is a function $q : T^* \to 2^A$ that, given a time step $t \in T^*$, returns the subset $q(t) \subseteq A$ of arcs in G that are affected by a scheduled maintenance operation through one or several such *interventions*. We therefore denote by G_t the subgraph of G with set $A \setminus q(t)$ of available arcs, *i.e.* taking into account the intervention scenario that takes down some links.

Budget For a given network and a set of demands, the routing scheme may require some changes or reconfiguration from one time period to the other, in order to take into account the links and nodes that are down. Each change in a routing scheme may require a human action that has a cost. In this context, it is desirable to limit the number of network reconfigurations (i.e. adding/removing waypoints). We denote by $\mathrm{dist}(P, P') \in \mathbb{N}$ the number of changes between two routing schemes P and P'. More formally,

$$\text{dist}(P, P') = \sum_{d \in D, i,j \in V, i \neq j} |\delta(p_d, ij) - \delta(p'_d, ij)|$$

where $\delta(p_d, ij)$ is equal to 1 if the segment path p_d associated to the demand d contains the segment (i, j), 0 otherwise (see Table 1).

For each time step, a budget function is given, denoted $\kappa : T^* \to \mathbb{N}$, that represents the maximum number of changes allowed from one time step to next one.

Routing Scheme P	**Routing Scheme** P'	**Distance** $\text{dist}(P, P')$
$\{\langle s, w, t\rangle\}$	$\{\langle s, t\rangle\}$	3
$\{\langle s, w_1, w_2, t\rangle\}$	$\{\langle s, w_1, t\rangle\}$	3
$\{\langle s, w_1, w_2, t\rangle\}$	$\{\langle s, w_2, w_1, t\rangle\}$	6
$\{\langle s, w_1, t\rangle\}$	$\{\langle s, w_2, t\rangle\}$	4
$\{\langle s_1, w_1, t_1\rangle, \langle s_2, w_2, w_3, t_2\rangle\}$	$\{\langle s_1, w_4, t_1\rangle, \langle s_2, w_3, w_2, t_2\rangle\}$	10 $(= 4 + 6)$

Table 1: Examples of distance values between several routing schemes

Load The load $\lambda(a, t) \in \mathbb{Q}^+$ of arc $a \in A$ at time step $t \in T$, is the ratio between the quantity of flow using an arc and its capacity. This ratio is also referred to as *link utilization* and is often used as (part of) the optimization criterion in the literature related to Traffic Engineering. Note that it can be higher than 1 in case of congestion on the link. In order to compute the load, we introduce the concept of *split coefficients*, denoted by $r(u, v, a, t)$. They represent the proportion of flow between source node u and target node v that passes through arc a of $FG(u, v)$ under the condition of the network at time t (i.e. in graph G_t). The ratios are given for all couples of nodes (u, v) because the segment (u, v) can be potentially used for routing a demand. Note that $r(u, v, a, t)$ is always greater or equal to 0, and $r(u, v, a, t) = 0$ if arc a is not in the forwarding graph $FG(u, v)$. The load of an arc $a \in A$ at time step $t \in T$ is then:

$$\lambda(a, t) = \frac{\sum_{d \in D} \sum_{i,j \in V} r(i, j, a, t)\ \nu(d, t)\ \delta(p_d, ij)}{c(a)}$$

See Figure 1 for an illustration of ECMP, split coefficients and of a segment path with one waypoint.

Maximum Link Utilization (MLU) Given a network $(G = (V, E), \omega, c)$, a set of k demands $D = \{(s_i, t_i) : i = 1, \ldots, k\}$ on G, a *routing scheme* P for D, the maximum link utilization, denoted $\text{mlu}(P, D, G) \in \mathbb{Q}^+$ is the load of the most loaded link.

3 Problem description

In this section, we introduce the challenge problem called T-Adaptive Segment Routing (T-ASR). A formal expression of the decision variables, constraints and objective function of T-ASR is provided.

3.1 Decisions

Routing Let us define the binary variable x_{ij}^{dt} that takes value 1 if the segment path of demand $d \in D$ contains segment (i, j) at time step $t \in T$ and 0 otherwise.
Therefore, a solution of the T-ASR problem corresponds to a *routing scheme* induced by the variables x with non-zero values, that minimizes the objective described in Section 3.3, and satisfies the constraints given in Section 3.2, for each period of time.

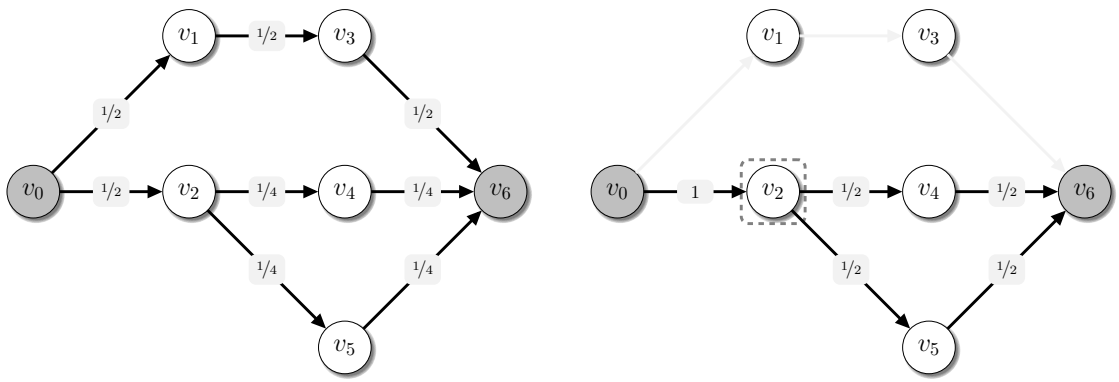

Figure 1: On the left, the demand (v_0, v_6) is routed on a network with unit weights through the segment path $\langle v_0, v_6 \rangle$. The associated forwarding graph, $FG(v_0, v_6)$, includes all arcs. The fraction on each arc indicates how the flow is split among the shortest paths (split coefficient) using the ECMP rule. On the right, a waypoint in v_2 is introduced, resulting in the flow being routed through the segment path $\langle v_0, v_2, v_6 \rangle$ that is, through the bottom part of the network. The associated forwarding graphs, $FG(v_0, v_2)$ and $FG(v_2, v_6)$, are represented with bold arcs.

3.2 Constraints

Flow-conservation constraints The following set of equalities ensures that each traffic demand $d \in D$ is routed along one segment path which connects its origin and its destination, at each time step t, *i.e.*,

$$\sum_{j \in V \setminus \{i\}} x_{ij}^{dt} - \sum_{j \in V \setminus \{i\}} x_{ji}^{dt} = \begin{cases} 1 & \text{if } i = s, \\ -1 & \text{if } i = t, \\ 0 & \text{otherwise.} \end{cases} \quad \begin{array}{l} \forall i \in V, \\ \forall d = (s,t) \in D, \forall t \in T \end{array} \tag{1}$$

Number of segments The following inequalities allow, for each demand d and each time step t, to limit the number of segments, including the first hop from the source node of d to the first waypoint visited, and the last waypoint visited to the destination node, used in the solution.

$$\sum_{i,j \in V} x_{ij}^{dt} \leq \texttt{maxSeg}, \qquad \forall d \in D, \forall t \in T \tag{2}$$

Load constraints The total traffic of a link is computed by adding the traffic of all demands that are routed through this link.

$$\sum_{d \in D} \sum_{i,j \in V} r(i,j,a,t)\ \nu(d,t)\ x_{ij}^{dt} \leq \lambda(a,t)\ c(a), \qquad \forall t \in T, \forall a \in A \setminus q(t) \tag{3}$$

Budget constraints Since each network configuration change incurs significant operational costs and may also risk deteriorating the Quality of Service, it is desirable to incorporate budget limitations in the model. The following set of inequalities (4) ensure that the value returned by dist (defined in Section 2, paragraph *Budget*) is indeed bounded by κ and allow to restrict the number of waypoint changes scheduled from one time step to the next.

$$\sum_{d \in D} \sum_{i,j \in V, i \neq j} |x_{ij}^{dt} - x_{ij}^{d,t-1}| \leq \kappa(t), \qquad \forall t \in T^* \tag{4}$$

3.3 Objective

Among the various objective functions discussed in the state-of-the-art of mathematical optimization for Traffic Engineering, the most commonly used is to minimize the load of the most heavily loaded link. This approach is extended by minimizing the load of all arcs across all time steps, by decreasing lexicographic order. It means that the aim is to minimize the most loaded link, then the second most loaded link, and so on. This leads to the following formulation for the problem T-ASR.

Let $L = \{\lambda(a,t) \mid a \in A, t \in T\}$ be the set of loads of all arcs at all time steps.

$$\text{lex min } \{\lambda'(i) : 1 \leq i \leq |L|\} \quad \text{where } \lambda'(i) \text{ is the i-th largest element of } L \tag{5}$$

$$(1) - (4), \tag{6}$$

$$x_{ij}^{dt} \in \{0,1\}, \qquad \forall d \in D, \forall t \in T, \forall i \in V, j \in V \setminus \{i\}, \tag{7}$$

$$\lambda' : \{1, 2, \ldots, |L|\} \to \mathbb{Q}^+. \tag{8}$$

3.4 Problem definition

The formal definition of the considered optimization problem is

T-Adaptive Segment Routing (T-ASR)

Input: A network $(G = (V, A), \omega, c)$, a set of demands $D = \{(s_i, t_i) : i = 1, \ldots, k\}$, a traffic volume $\nu : D \times T \to \mathbb{Q}$, set of time periods $T = \{0, 1, \ldots, h-1\}$ with horizon $h \geq 1$, an intervention scenario $q : T^* \to 2^A$, a budget $\kappa : T^* \to \mathbb{N}$ and a maximum number of segments `maxSeg` $\in \mathbb{N}$ (`maxSeg` ≥ 2).
Output: A routing scheme P^t for D for each time step $t \in T$ such that the constraints (1) - (4) are satisfied, and the objective function (5) is minimized.

4 Example of instance

In this subsection, a complete instance is described, with all the input parameters.

4.1 Network

Figure 2 illustrates the input network for an instance (named hereafter 'toy'), that is also used as example in Section 5 about instance file formats.

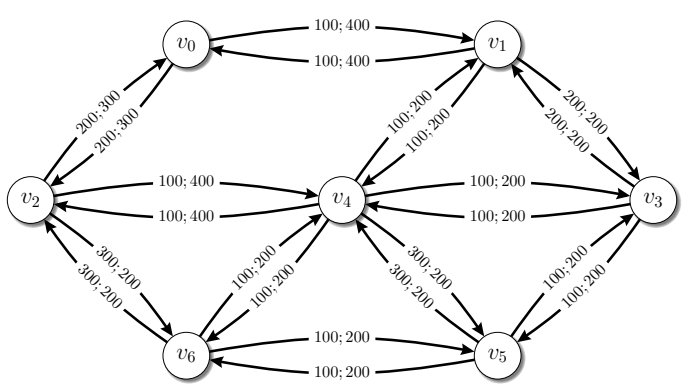

Figure 2: Network (V, A, ω, c) for the 'toy' instance. A label "$\omega(a); c(a)$" is associated to each arc $a \in A$.

4.2 Demands

From the graph (V, A) and the weight function ω, we can derive the values of $r(u, v, a, t)$, for each couple of nodes $(u, v) \in V \times V$, each arc $a \in A$, and each time step $t \in T$. In this example, we consider two demands:

- from source v_0 to target v_5
- from source v_2 to target v_5

The values of $r(u, v, a, t)$ at $t = 0$ are given in Table 2, computed as explained in Section 2, paragraph *Load*.

Arcs / Demands	**Demand (v_0,v_5)**	**Demand (v_2,v_5)**
Arc $a_{0,1}$	$r(v_0, v_5, a_{0,1}, 0) = 1$	$r(v_2, v_5, a_{0,1}, 0) = 0$
Arc $a_{0,2}$	$r(v_0, v_5, a_{0,2}, 0) = 0$	$r(v_2, v_5, a_{0,2}, 0) = 0$
Arc $a_{1,3}$	$r(v_0, v_5, a_{1,3}, 0) = 0.5$	$r(v_2, v_5, a_{1,3}, 0) = 0$
Arc $a_{1,4}$	$r(v_0, v_5, a_{1,4}, 0) = 0.5$	$r(v_2, v_5, a_{1,4}, 0) = 0$
Arc $a_{2,4}$	$r(v_0, v_5, a_{2,4}, 0) = 0$	$r(v_2, v_5, a_{2,4}, 0) = 1$
Arc $a_{2,6}$	$r(v_0, v_5, a_{2,6}, 0) = 0$	$r(v_2, v_5, a_{2,6}, 0) = 0$
Arc $a_{3,5}$	$r(v_0, v_5, a_{3,5}, 0) = 0.75$	$r(v_2, v_5, a_{3,5}, 0) = 0.5$
Arc $a_{4,3}$	$r(v_0, v_5, a_{4,3}, 0) = 0.25$	$r(v_2, v_5, a_{4,3}, 0) = 0.5$
Arc $a_{4,5}$	$r(v_0, v_5, a_{4,5}, 0) = 0$	$r(v_2, v_5, a_{4,5}, 0) = 0$
Arc $a_{4,6}$	$r(v_0, v_5, a_{4,6}, 0) = 0.25$	$r(v_2, v_5, a_{4,6}, 0) = 0.5$
Arc $a_{6,5}$	$r(v_0, v_5, a_{6,5}, 0) = 0.25$	$r(v_2, v_5, a_{6,5}, 0) = 0.5$

Table 2: Values of split coefficients $r(u, v, a, t)$ at $t = 0$

For Demand (v_0, v_5) the split coefficients come directly from the fact that the flow is split between three shortest paths: ($a_{0,1}$, $a_{1,3}$, $a_{3,5}$), ($a_{0,1}$, $a_{1,4}$, $a_{4,6}$, $a_{6,5}$) and ($a_{0,1}$, $a_{1,4}$, $a_{4,3}$, $a_{3,5}$). For Demand (v_2, v_5) there are two shortest paths: ($a_{2,4}$, $a_{4,6}$, $a_{6,5}$) and ($a_{2,4}$, $a_{4,3}$, $a_{3,5}$).

The demands are associated to traffic volumes according to Table 3.

Time steps / Demands	**Demand (v_0,v_5) = d_0**	**Demand (v_2,v_5) = d_1**
Time step 0	$\nu(d_0, 0) = 50$	$\nu(d_1, 0) = 100$
Time step 1	$\nu(d_0, 1) = 100$	$\nu(d_1, 1) = 50$

Table 3: Values of traffic volumes $\nu(d, t)$

By combining the values of Table 2 and 3 with the arc capacities, we can compute the loads of arcs. For instance the load of arc $a_{4,6}$ at time step $t = 0$ is:

$$\lambda(a_{4,6}, 0) = \frac{r(v_0, v_5, a_{4,6}, 0)\nu(d_0, 0) + r(v_2, v_5, a_{4,6}, 0)\nu(d_1, 0)}{c(a_{4,6})} = \frac{0.25 * 50 + 0.5 * 100}{200} = 0.3125$$

4.3 Intervention Scenario

At time step 1, let us assume that there is an intervention on the links $a_{1,3}$ and $a_{1,4}$. Figure 3 shows the impact of this intervention on the flow (blue lines) of the first demand (v_0, v_5) at time step 0 and 1.

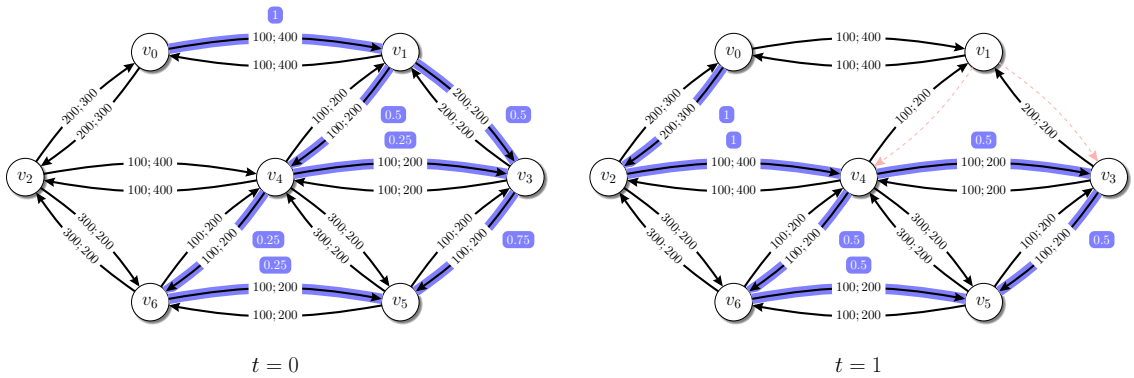

Figure 3: Illustration of the flow of demand (v_0, v_5) for the 'toy' instance at time step $t = 0$ (left) and time step $t = 1$ (right) where links $a_{1,3}$ and $a_{1,4}$ are down.

4.4 Solutions

A solution is characterized by a list of waypoints for each demand and each time step. Note that the list can be empty for some demand/time step. The waypoints determine the segment paths that are used.

In the toy example, a trivial solution is made by using no segment path. In this case, the resulting loads are given in Figure 4 (only the non-zero loads are given).

We can now consider another solution in which there is a waypoint v_4 for demand (v_0,v_5) at time step 0. In that case, we can compute the loads, as given in Figure 4.

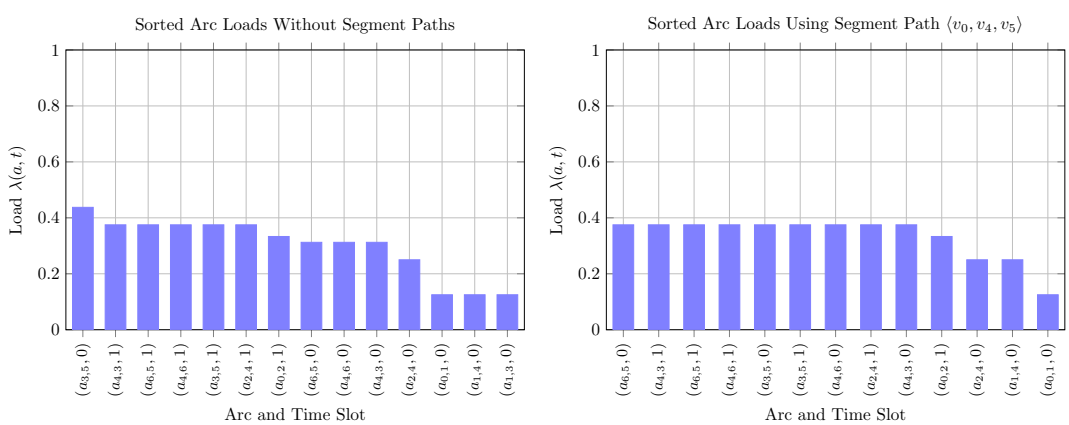

Figure 4: Non-zero arc loads sorted in decreasing order induced by the trivial solution consisting of using no segment paths (left) and the solution that uses the segment path $\langle v_0, v_4, v_5 \rangle$ for demand (v_0, v_5) at $t = 0$ (right). The highest load is reduced from 0.4375 to 0.3750. Note that the reduction in the maximum load is accompanied by load increases on several other arcs.

The solution with a waypoint used is better than the trivial solution because the highest load is reduced from 0.4375 to 0.3750.

5 Input/Output File Formats

5.1 Input files

Each instance of the problem is defined by the following three JSON files, which together describe the network, the traffic demands, and the intervention scenarios:

Network Topology — a NetworkX [2] compatible JSON file describing the physical network, including nodes and links with associated attributes.

Traffic Matrix — a custom JSON file specifying traffic demands between nodes over time.

Intervention Scenario — a custom JSON file detailing interventions occurring at specific time steps, along with budget constraints.

In the following sections, we give a representative example of each file format used in a typical instance.

5.1.1 Network Topology: `toy-net.json`

It is a JSON file and it contains two main sections: `nodes` and `links`. The file describes a directed graph with attributes on nodes and edges. The graph is not a multigraph (i.e., no multiple edges between the same pair of nodes).

Attribute	Type	Example	Comments
`directed`	boolean	true	Indicates whether the graph is directed.
`multigraph`	boolean	false	Indicates whether multiple arcs between nodes are allowed.
Nodes section			
`name`	string	"B"	Name of the node (label).
`id`	integer	1	Unique identifier of the node.
Links section			
`id`	integer	17	Unique identifier of the link
`metric`	float	100	Metric value (i.e. arc weight).
`capacity`	float	400	Maximum capacity of the link.
`from`	integer	0	ID of the source node.
`to`	integer	1	ID of the destination node.

Each node is defined by a unique ID and a name. Each link connects two nodes and includes attributes such as the `metric` (i.e. weight) and the `capacity`. All node IDs referenced in the `from` and `to` fields must be defined in the `nodes` section.

toy-net.json

```
{
  "directed": true,
  "multigraph": false,
  "nodes": [
    {"name": "A", "id": 0},
    {"name": "B", "id": 1},
    {"name": "C", "id": 2},
    {"name": "D", "id": 3},
    {"name": "E", "id": 4},
    {"name": "F", "id": 5},
    {"name": "G", "id": 6}
  ],
  "links": [
    {"id": 0, "from": 0, "to": 1, "metric": 100, "capacity": 400},
    {"id": 1, "from": 0, "to": 2, "metric": 200, "capacity": 300},
    {"id": 2, "from": 1, "to": 3, "metric": 200, "capacity": 200},
    {"id": 3, "from": 1, "to": 4, "metric": 100, "capacity": 200},
    {"id": 4, "from": 2, "to": 4, "metric": 100, "capacity": 400},
    {"id": 5, "from": 2, "to": 6, "metric": 300, "capacity": 200},
    {"id": 6, "from": 3, "to": 5, "metric": 100, "capacity": 200},
    {"id": 7, "from": 3, "to": 4, "metric": 100, "capacity": 200},
```

```
    {"id": 8, "from": 4, "to": 5, "metric": 300, "capacity": 200},
    {"id": 9, "from": 4, "to": 6, "metric": 100, "capacity": 200},
    {"id": 10, "from": 5, "to": 6, "metric": 100, "capacity": 200},
    {"id": 11, "from": 1, "to": 0, "metric": 100, "capacity": 400},
    {"id": 12, "from": 2, "to": 0, "metric": 200, "capacity": 300},
    {"id": 13, "from": 3, "to": 1, "metric": 200, "capacity": 200},
    {"id": 14, "from": 4, "to": 1, "metric": 100, "capacity": 200},
    {"id": 15, "from": 4, "to": 2, "metric": 100, "capacity": 400},
    {"id": 16, "from": 6, "to": 2, "metric": 300, "capacity": 200},
    {"id": 17, "from": 5, "to": 3, "metric": 100, "capacity": 200},
    {"id": 18, "from": 4, "to": 3, "metric": 100, "capacity": 200},
    {"id": 19, "from": 5, "to": 4, "metric": 300, "capacity": 200},
    {"id": 20, "from": 6, "to": 4, "metric": 100, "capacity": 200},
    {"id": 21, "from": 6, "to": 5, "metric": 100, "capacity": 200}
  ]
}
```

5.1.2 Traffic Matrix: `toy-tm.json`

The file describes a directed graph with demand values over multiple time steps.

Field	Type	Example	Comments
`num_time_slots`	integer	2	Number of time steps over which demand is defined.
`v (value)`	array of floats	[50, 100]	Traffic values for each time slot.
`s (source)`	integer	0	ID of the source node.
`t (target)`	integer	5	ID of the destination node.

All node IDs referenced in the `s` (source) and `t` (target) fields must be defined in the `nodes` section of the network input file.

toy-tm.json

```
{
  "num_time_slots" : 2,
  "demands": [
    { "v": [ 50.0, 100.0], "s": 0, "t": 5},
    { "v": [ 100.0, 50.0], "s": 2, "t": 5}
  ]
}
```

5.1.3 Interventions Scenario: `toy-scenario.json`

It is a JSON file that contains two main sections: `budget` and `interventions`. The file describes interventions scenario over a discrete time horizon, specifying which links will be deactivated at which time steps and the budget available at each time step.

Field	Type	Example	Comments
`max_segments`	integer	4	Maximum number of segments per segment path.
Budget section			
`t`	integer	1	Time index at which the budget applies.
`value`	integer	20	Maximum number of actions for this time step.
Interventions section			
`t`	integer	1	Time index at which the intervention occurs.
`links`	array of integers	[2, 3]	List of link IDs that are offline at the given time step.

Each intervention entry specifies a time step and the list of link IDs that are considered offline at that time. The `budget` array defines the available budget for each time step (in the time horizon), which may be used to mitigate or respond to interventions.

As a reminder, **at time step 0, the budget can be considered infinite and there is no intervention** (i.e. no down link). This is the reason why time step 0 is not mentioned in the scenario input file.

toy-scenario.json

```
{
  "max_segments": 4,
  "budget": [
    { "t": 1, "value": 20 }
  ],
  "interventions": [
    { "t": 1, "links": [2, 3] }
  ]
}
```

5.2 Output

5.2.1 Segment Routing Paths (SR-paths): `toy-srpaths.json`

This file represents a solution of the instance. It specifies the waypoints for each demand and each time step. These waypoints determine the segment paths that will be used. When a waypoint list is empty, the entry with the corresponding demand and time step can be omitted from the file.

Field	Type	Example	Comments
`d (demand)`	integer	1	ID of demand
`t (time)`	integer	2	time step
`w (waypoints)`	array of integers	[2, 3]	List of waypoints (node IDs) in the segment path

toy-srpaths.json

```
{
    "srpaths": [
        { "d": 0, "t": 0, "w": [3] },
        { "d": 0, "t": 1, "w": [3] },
        { "d": 1, "t": 0, "w": [6] },
        { "d": 1, "t": 1, "w": [6] }
    ]
}
```

Bibliography

[1] Clarence Filsfils, Stefano Previdi, Les Ginsberg, Bruno Decraene, Stephane Litkowski, and Rob Shakir. *Segment Routing Architecture*. RFC 8402. July 2018. URL: https://www.rfc-editor.org/info/rfc8402.

[2] Aric Hagberg, Pieter J Swart, and Daniel A Schult. "Exploring network structure, dynamics, and function using NetworkX". In: *Proceedings of the 7th Python in Science Conference (SciPy2008)*. 2008, pp. 11–15.

A Table of notations

Notation	Description
Network & Graph	
$G = (V, A)$	Directed graph representing the network, with vertex set V and arc set A.
$\omega(a)$	Weight function for arc $a \in A$, used for shortest path computations.
$c(a)$	Capacity (bandwidth) of arc $a \in A$.
$FG(u, v)$	Forwarding graph containing arcs on shortest paths from u to v.
$r(u, v, a, t)$	Split coefficient: fraction of flow from u to v traversing arc a at time t.
$\lambda(a, t)$	Load (link utilization) of arc a at time t.
mlu	Maximum Link Utilization.
Time & Interventions	
T	Set of time periods $\{0, 1, \ldots, h-1\}$.
T^*	Set of time periods excluding the initial nominal state, $T^* = T \setminus \{0\}$.
$q(t)$	Intervention scenario: subset of arcs unavailable at time t.
G_t	Subgraph of G with available arcs $A \setminus q(t)$ at time t.
$\kappa(t)$	Maximum budget for configuration changes at time t.
Demands & Segment Routing	
D	Set of traffic demands, where each demand $d = (s, t)$ has a source and target.
$\nu(d, t)$	Traffic volume for demand d at time t.
$\langle s, w_1, \ldots, t \rangle$	Notation for a segment path with waypoints w_i.
`maxSeg`	Maximum number of segments allowed in a path.
P	Routing scheme (set of segment paths for all demands).
$\text{dist}(P, P')$	Distance between two routing schemes (number of changes).
$\delta(p_d, ij)$	Indicator equal to 1 if path p_d uses segment (i, j).
Decision Variables	
x_{ij}^{dt}	Binary variable: 1 if the path for demand d uses segment (i, j) at time t.

Table 4: Table of Notations